Project:
Student Dropout Prediction Analysis Report

Phoebe Voong-Fadel

Confidentiality Note:

This report is an anonymised portfolio version of an academic machine learning case study. Organisation names, operational details, dataset schema, feature labels, and selected contextual references have been generalised or removed to protect confidential information while preserving the modelling approach, evaluation logic, and core findings.

Abbreviations

AUC — Area Under the Curve

EDA — Exploratory Data Analysis

ML — Machine Learning

NN — Neural Network

ROC Curve — Receiver Operating Characteristic Curve

XGBoost — Extreme Gradient Boosting

Introduction

This project investigates how supervised machine learning can be used to predict student dropout risk across multiple stages of the student journey.

Student retention is an important issue in education because it affects learner outcomes, operational planning, and the effectiveness of support interventions. The aim of this analysis is to evaluate whether dropout risk can be identified early enough to support timely action, and whether predictive performance improves as richer data becomes available over time.

The project is structured in three stages. The first stage uses early applicant and enrollment information. The second stage introduces engagement-related variables. The third stage adds academic progress variables. This staged design allows comparison of how predictive performance changes as more information is introduced.

Data Analysis and Preprocessing

The analysis followed a consistent exploratory and preprocessing workflow across all three stages. Initial review focused on dataset structure, duplicates, missing values, feature types, and cardinality. Variables with substantial missingness were removed where appropriate, and identifier columns were excluded because they had no predictive value.

A date-based field was transformed into an age variable to create a more meaningful numeric feature. The target variable was encoded into binary form to support classification modelling. Ordered categorical variables were encoded using a predefined ranking, while nominal categorical variables were one-hot encoded. Data was then split into training, validation, and test sets using a stratified approach to preserve class balance.

In the second stage, additional engagement-related features were introduced. Exploratory analysis suggested that these variables added useful signal, and a small proportion of records with missing values was handled during preprocessing. The remaining workflow was then applied consistently with Stage 1.

In the third stage, additional academic progress variables were added. These variables appeared to have stronger association with the target outcome than the earlier feature sets. Missing values at this stage were addressed using standard imputation methods, fitted on the training set only to avoid data leakage. The remaining preprocessing mirrored earlier stages.

Methodology

Overview

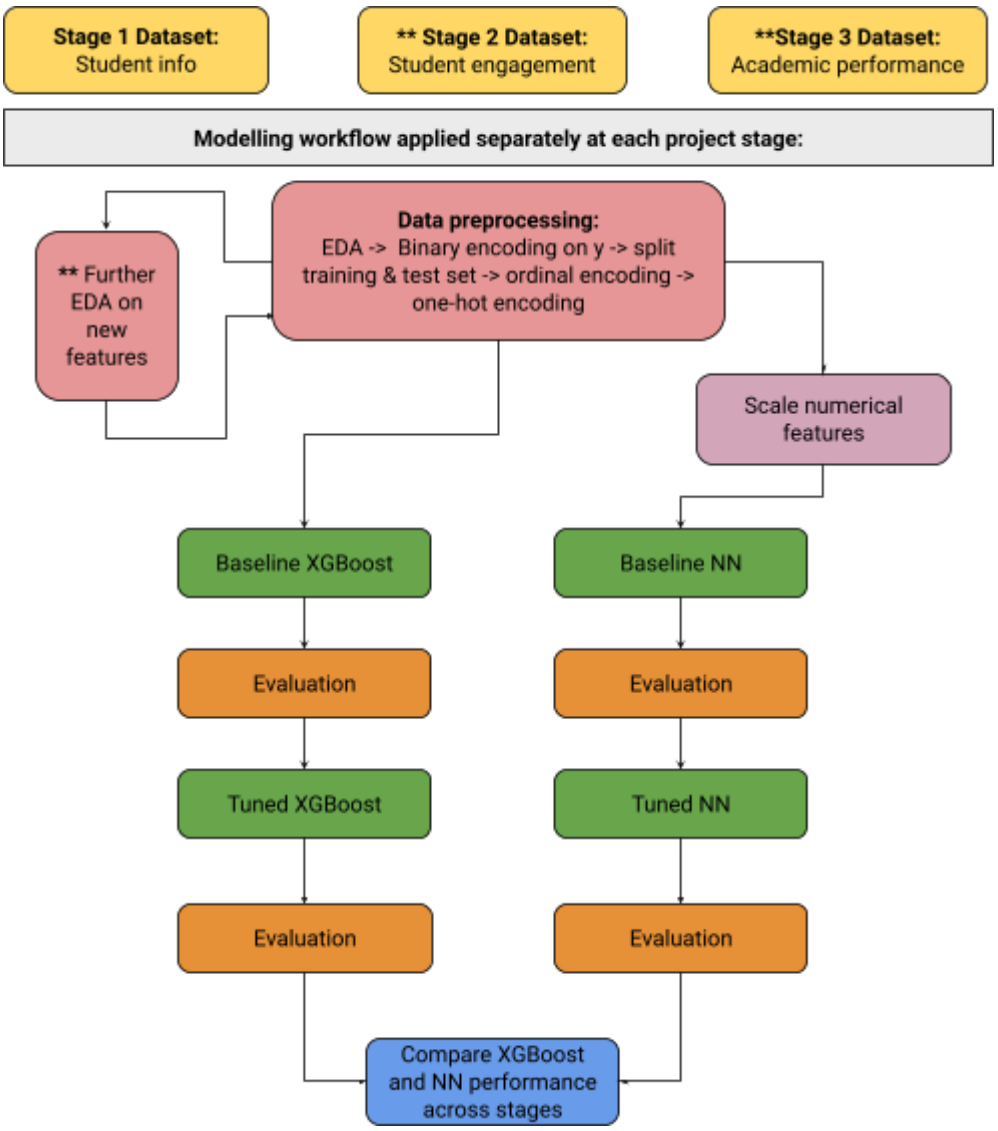


Figure 2: Overview of the modelling methodology used across the three project stages.

This project used supervised ML (XGBoost and NN) to predict course completion across three journey stages. As illustrated in Figure 2, the consistent workflow involved EDA and preprocessing, including binary target encoding, ordinal encoding for '`CourseLevel`', and one-hot encoding for categorical variables. Data was partitioned via a stratified 80:20 train-test split, reserving 10% of training data for validation. Finally, numerical features were standardised specifically for the NN model.

Model specification

A baseline XGBoost classifier was trained using the library's default core model hyperparameters (XGBoost Developers, 2026), with `random_state` fixed for reproducibility, `eval_metric='logloss'`, and early stopping applied using the validation set. XGBoost tuning explored `learning_rate`, `max_depth`, and `n_estimators` using GridSearchCV.

For the NN, a feedforward architecture with hidden layers of 128, 64, 32 and 16 neurons, `ReLU` activations, dropout (0.2), and a `sigmoid` output layer was used as the baseline, with tuning applied to layer sizes, activation function, optimiser, and learning rate using Hyperband (O'Malley et al., 2026).

Further model specification details are provided in Appendix B.

Workflow

At each stage, baseline XGBoost and NN models were trained and hyperparameter-tuned. Test set performance was evaluated using accuracy, precision, recall, ROC-AUC, PR curves, and confusion matrices. This multi-metric approach ensured a balanced assessment, addressing class imbalance more effectively than accuracy alone. Finally, performance was compared across Stages 1–3 to analyse how predictive accuracy evolved as additional student data became available.

Results

Baseline model performance by stage

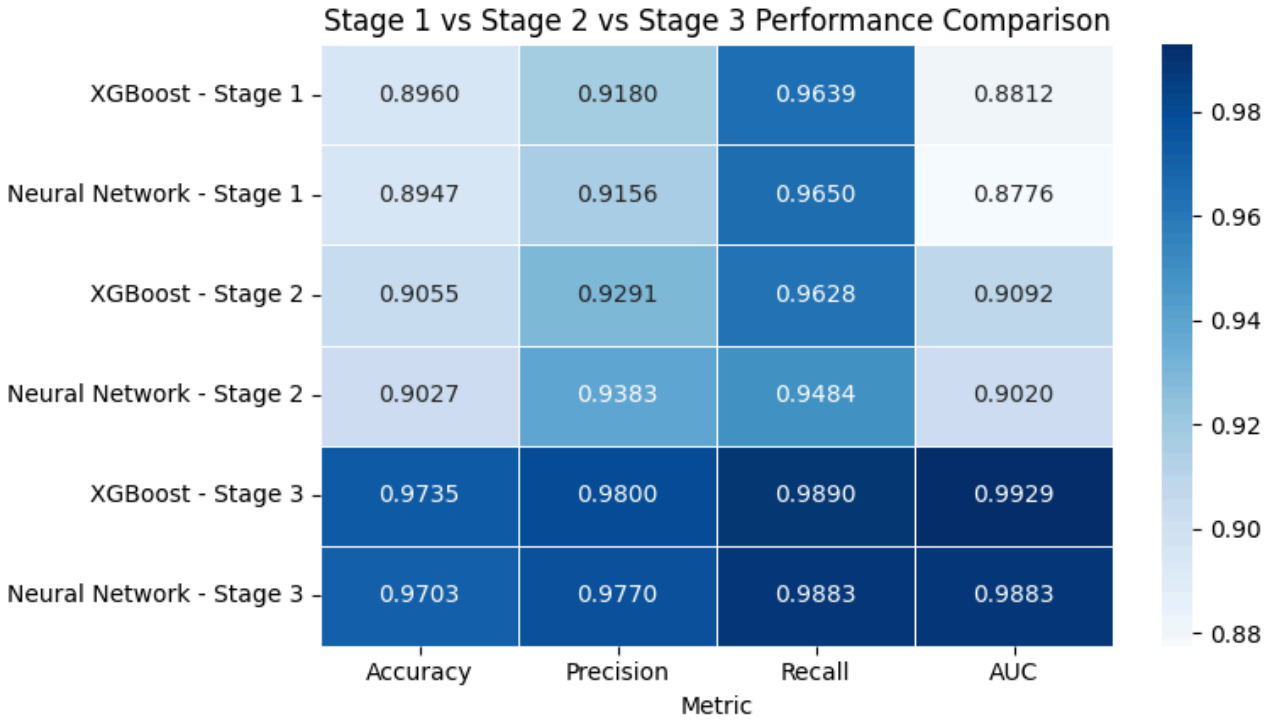


Figure 3: Baseline model performance across Stages 1–3

Baseline performance improved across each stage for both models (see Figure 3). XGBoost performed slightly better than the NN at every stage for accuracy, with the strongest overall results achieved in Stage 3. Change in baseline performance across stages

Model	Accuracy	Precision	Recall	AUC
XGBoost: **Stage 1 → 2**	+0.0095	+0.0111	-0.0011	+0.0280
NN: **Stage 1 → 2**	+0.0080	+0.0227	-0.0166	+0.0244
XGBoost: **Stage 2 → 3**	+0.0680	+0.0509	+0.0262	+0.0837
NN: **Stage 2 → 3**	+0.0676	+0.0387	+0.0399	+0.0863

Table 2: Change in baseline model performance across stages.

The improvement from Stage 1 to 2 was modest (Table 2), indicating that attendance variables added useful but limited signals. In contrast, the improvement from Stage 2 to 3 was substantial for both models.

Neural Network training behaviour

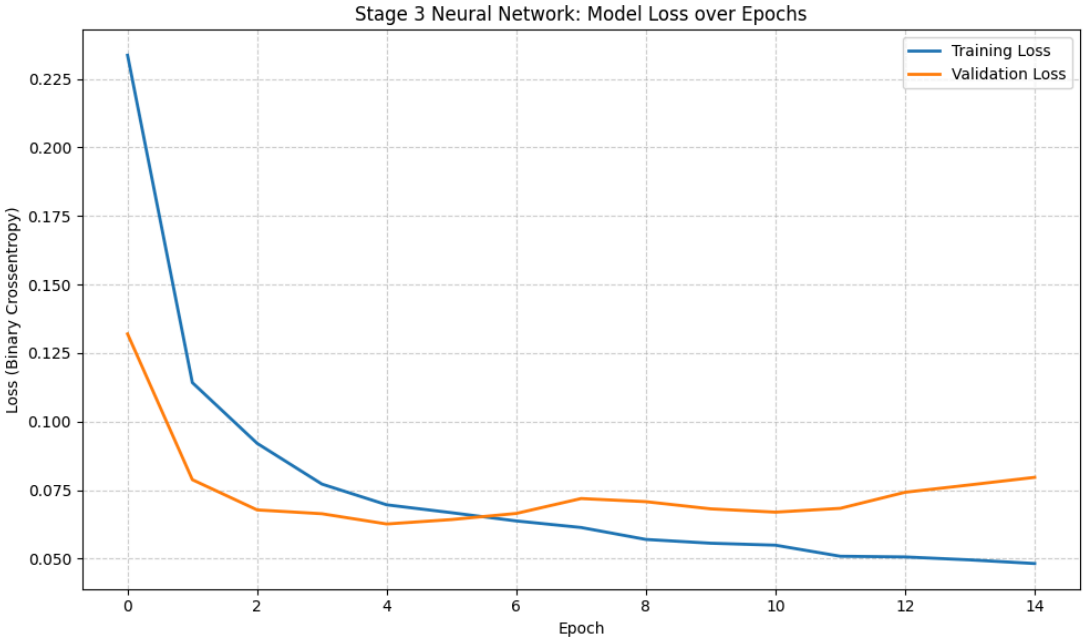


Figure 5: Baseline NN training and validation loss over epochs for Stage 3.

The baseline NN loss curves indicate mild overfitting across the stages, as validation loss begins to rise after an initial improvement while training loss continues to decrease. This pattern is most evident in Stages 1 and 2 (refer to appendix C), whereas Stage 3 (figure 5) achieves lower loss overall, consistent with its stronger predictive performance.

Tuned model performance by stage

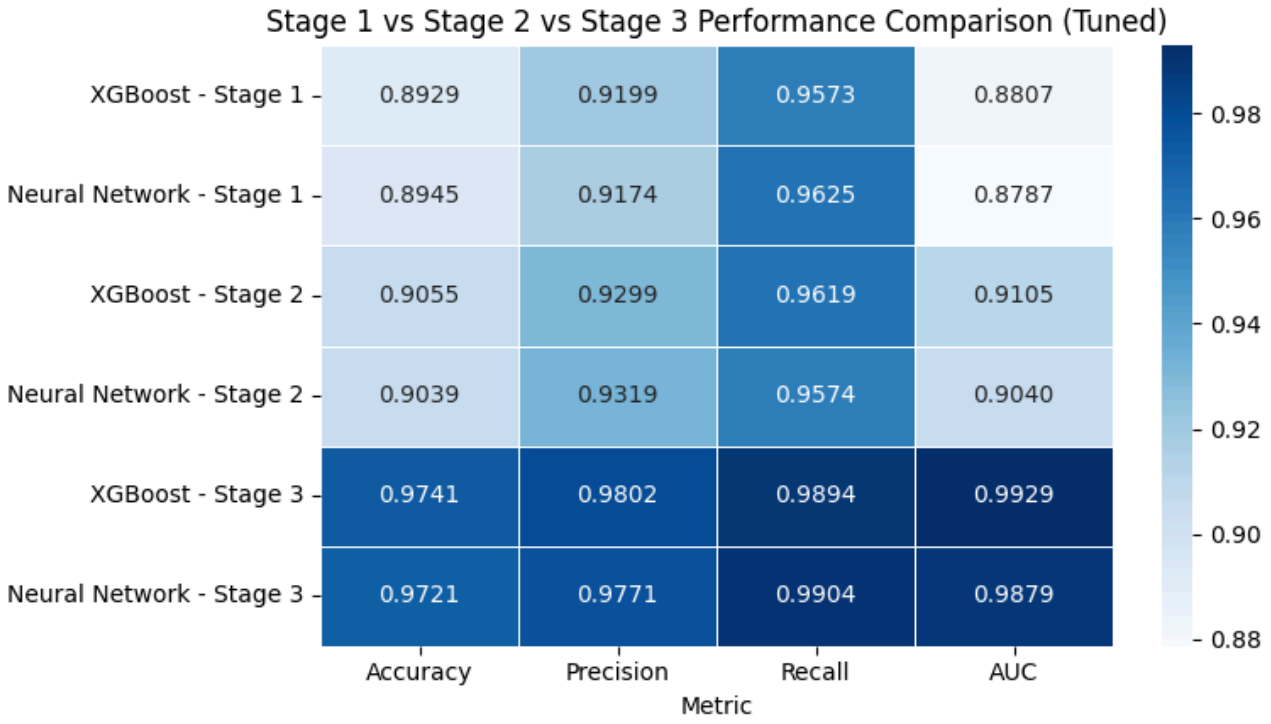


Figure 6: Tuned model performance across the stages 1 - 3.

Tuned models followed baseline trends (Figure 6), with performance rising slightly in Stage 2 and sharply in Stage 3. XGBoost was the strongest overall (Accuracy: 0.9741, AUC: 0.9929). This suggests that Stage 3 academic variables were the primary performance drivers, while tuning provided only marginal gains over the baseline.

Change in tuned performance across stages

Model	Accuracy	Precision	Recall	AUC
XGBoost: Stage 1 → 2	+0.0126	+0.0100	+0.0046	+0.0298
NN: Stage 1 → 2	+0.0094	+0.0145	-0.0051	+0.0253
XGBoost: Stage 2 → 3	+0.0686	+0.0503	+0.0275	+0.0824
NN: Stage 2 → 3	+0.0682	+0.0452	+0.0330	+0.0839

Table 3: Change in tuned model performance across stages.

Across the stages, hyperparameter tuning produced only marginal changes in Accuracy, Precision, Recall and AUC, suggesting that the baseline models were already close to optimal.

Appendix C includes additional results and visualisations.

Conclusion

The results show that student dropout risk can be modelled at multiple stages of the learner journey, although predictive performance improves as richer data becomes available. Early-stage data can support initial risk screening, mid-stage data adds some useful behavioural signal, and later-stage academic variables provide the strongest predictive power.

Across all stages, XGBoost outperformed the Neural Network slightly. A likely explanation is that tree-based models are often well suited to structured tabular data, while the Neural Network may have been constrained by the relatively narrow tuning space used in this project.

Future work could extend this analysis by testing alternative model families, broadening the search space for neural network tuning, and exploring additional techniques for handling class imbalance. These steps may improve generalisability and strengthen the usefulness of the model for practical support interventions.

References

XGBoost Developers (2026) XGBoost Parameters. [Online] Available at: https://xgboost.readthedocs.io/en/stable/parameter.html (Accessed: 15 March 2026).

O'Malley, T., Bursztein, E., Long, J., Chollet, F., Jin, H. and Invernizzi, L. (2026) *KerasTuner: Hyperband Tuner*. [Online] Available at: https://keras.io/keras_tuner/api/tuners/hyperband/ (Accessed: 15 March 2026).

Appendices

Appendix A: Preprocessing

A1. Common preprocessing steps across all stages

- Identifier fields were removed because they did not provide predictive value.
- Variables with substantial missingness were excluded where appropriate.
- A date-based feature was transformed into an age-related numeric variable, after which the original date field was removed.
- High-cardinality categorical variables were removed where they would have created excessive dimensionality after encoding.
- The target variable was converted into binary form for classification.
- The data was split into training, validation, and test sets.
- Stratified sampling was used to preserve the class distribution across splits.

A2. Encoding strategy

- An ordered categorical feature was encoded using a predefined ranking to preserve its ordinal structure.
- Remaining categorical variables were encoded using one-hot encoding.
- Validation and test matrices were aligned to the training feature space to ensure consistent input dimensions.

A3. Scaling strategy

- Feature scaling was applied only for the Neural Network.
- Numerical and ordinal variables were standardised using parameters fitted on the training set only.
- The set of scaled variables expanded across stages as additional numeric inputs were introduced.

A4. Missing data handling

- Missing data was assessed separately at each stage.
- Where missingness was limited, a small number of observations were removed.

- Where missingness was more substantive, standard imputation methods were used.
- All imputers were fitted on the training data only to prevent leakage into validation and test sets.

  -

Appendix B: Model specification and tuning

B1: Baseline XGBoost

- A baseline XGBoost classifier was trained for each stage using the `XGBClassifier` implementation.
- The model used the library's default core hyperparameters, with:
  - `random_state = 42`
  - `eval_metric = 'logloss'`
    `early_stopping_rounds = 10`
- Early stopping was applied using the validation set to reduce unnecessary training and limit overfitting.

B2: Tuned XGBoost

- Hyperparameter tuning for XGBoost was performed using GridSearchCV with 3-fold cross-validation.
- The following parameter grid was explored:
  - `learning_rate`: [0.01, 0.1, 0.2]
  - `max_depth`: [3, 5, 7]
  - `n_estimators`: [100, 200]
- Model selection was based on validation performance, and the tuned model was then evaluated on the held-out test set.

B3: Baseline Neural Network

- A feedforward Neural Network was implemented using Keras.
- The baseline architecture consisted of:
  hidden layers of 128, 64, 32, and 16 neurons
  - ReLU activation in hidden layers
  - dropout = 0.2 after the first three hidden layers
  - a sigmoid output layer for binary classification

- The model was compiled with:

- Adam optimiser
- `learning_rate = 0.001`
- `binary_crossentropy` loss
- `accuracy` as the training metric

- Early stopping was applied using validation loss with `patience = 10` and `restore_best_weights = True`.

B4: Tuned Neural Network

- Neural Network tuning was performed using Keras Tuner (Hyperband).
  The following hyperparameters were explored:
  - first hidden layer size: [64, 128, 256]
  - second hidden layer size: [32, 64, 128]
  - activation function: [`relu`, `tanh`, `sigmoid`]
  - optimiser: [`adam`, `rmsprop`]
  - learning rate: [0.01, 0.001, 0.0001]

- Hyperband optimisation used:
  - objective: validation AUC
  - `max_epochs = 15`
  - `factor = 3`
- The best hyperparameter combination was then retrained on the training data and evaluated on the test set.

B5: Evaluation metrics

- Model performance was assessed using:
  - Accuracy
  - Precision
  - Recall
  - Confusion matrix
  - ROC-AUC
  - Precision-Recall curves
- These metrics were used because the dataset was imbalanced, meaning that accuracy alone would not provide a complete view of model performance.

Appendix C. Supplementary results and visualisations

C1: Effect of Hyperparameter Tuning

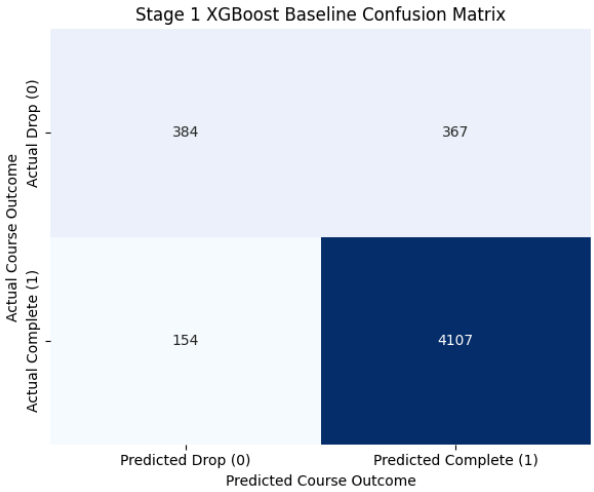
Stage	Model	Accuracy Change	Precision Change	Recall Change	AUC Change
Stage 1	XGBoost	-0.0032	+0.0020	-0.0066	-0.0005
Stage 1	NN	-0.0002	+0.0018	-0.0026	+0.0011
Stage 2	XGBoost	+0.0000	+0.0008	-0.0009	+0.0014
Stage 2	NN	+0.0012	-0.0063	+0.0089	+0.0020
Stage 3	XGBoost	+0.0006	+0.0002	+0.0005	+0.0001
Stage 3	NN	+0.0018	+0.0000	+0.0021	-0.0004

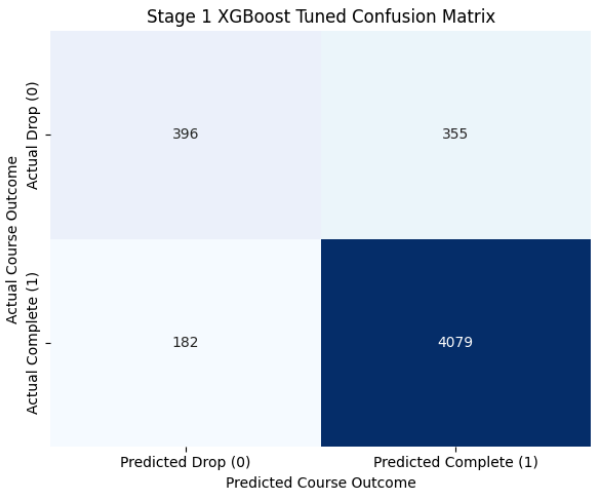
Table C1: Difference between baseline and tuned model performance across stages for NN and XGBoost.

Across all three stages, the differences between baseline and tuned models were very small (see Table C1). This indicates that hyperparameter tuning had limited impact on performance and in some cases it was marginally worse.

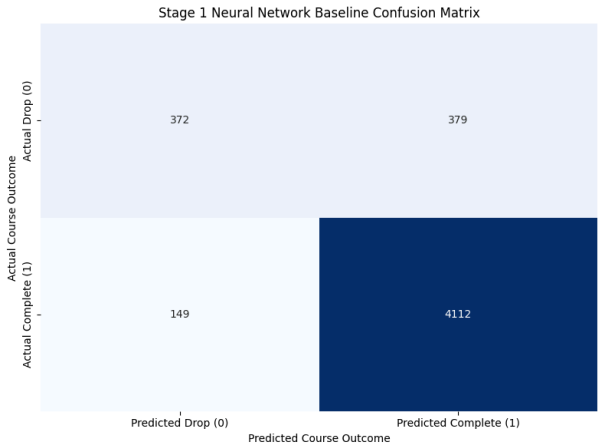
C2: Confusion matrices

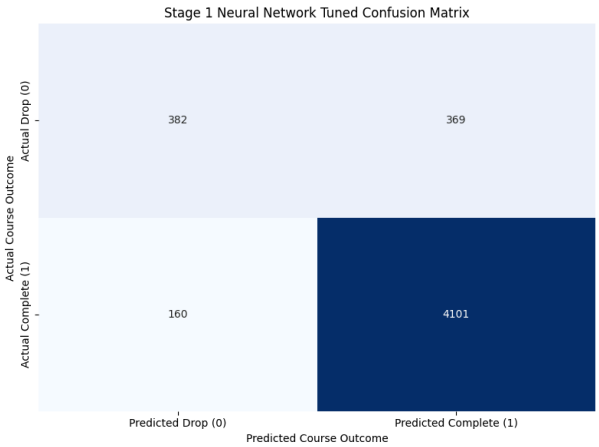
- Stage 1 XGBoost baseline and tuned confusion matrices



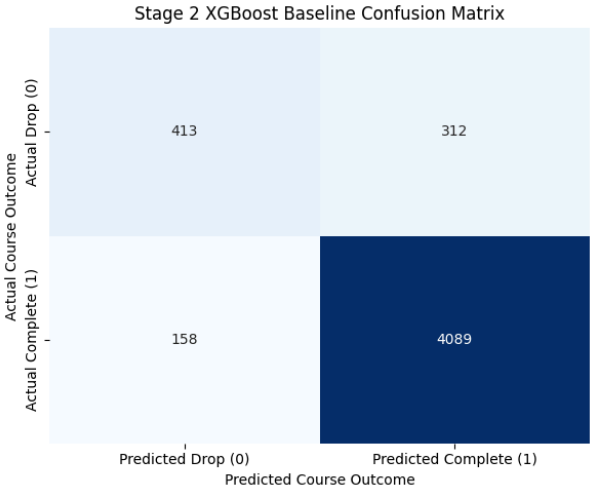


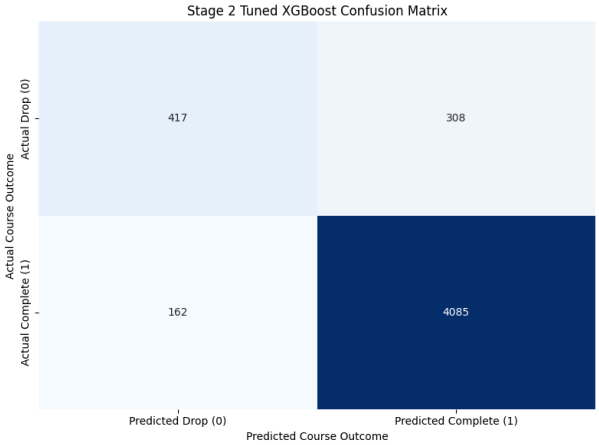
- Stage 1 Neural Network baseline and tuned confusion matrices



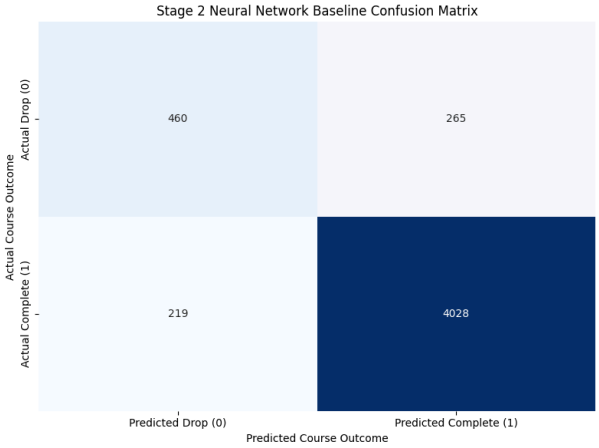


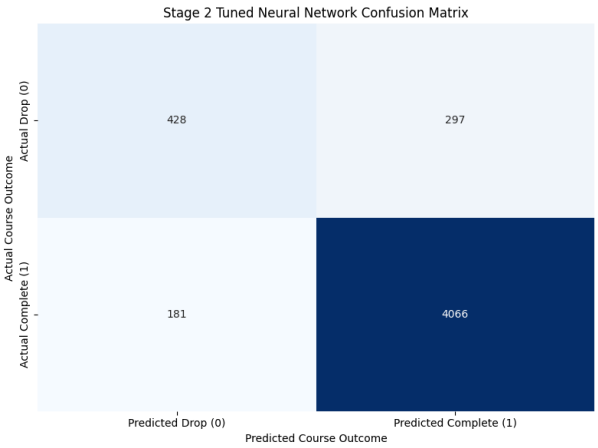
- Stage 2 XGBoost baseline and tuned confusion matrices



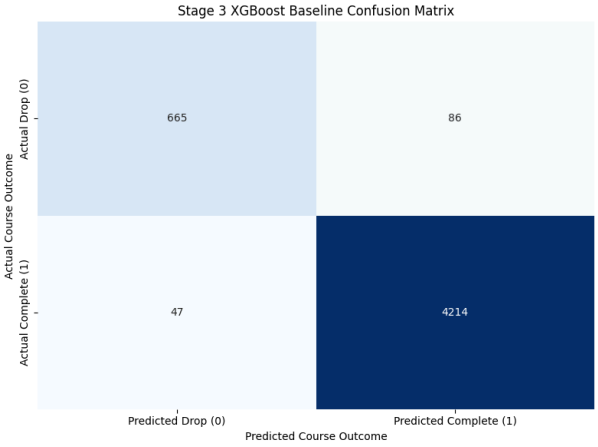


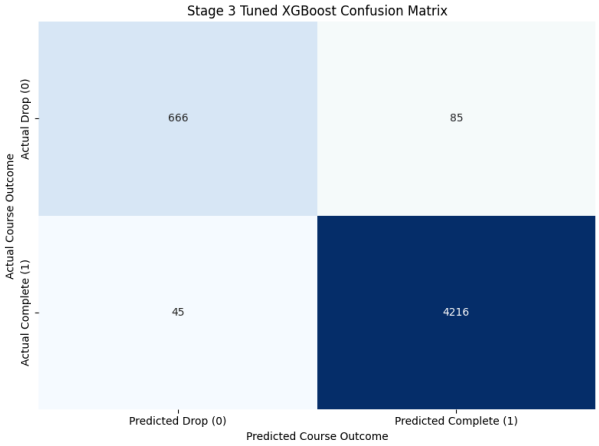
- Stage 2 Neural Network baseline and tuned confusion matrices



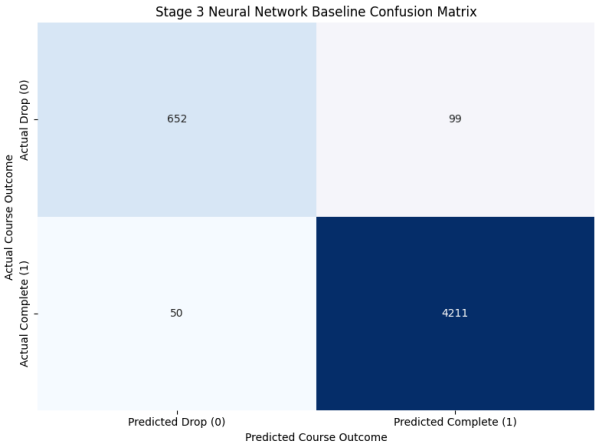


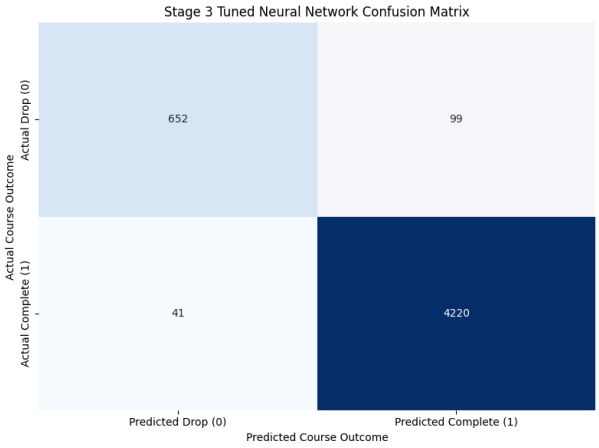
- Stage 3 XGBoost baseline and tuned confusion matrices





- Stage 3 Neural Network baseline and tuned confusion matrices



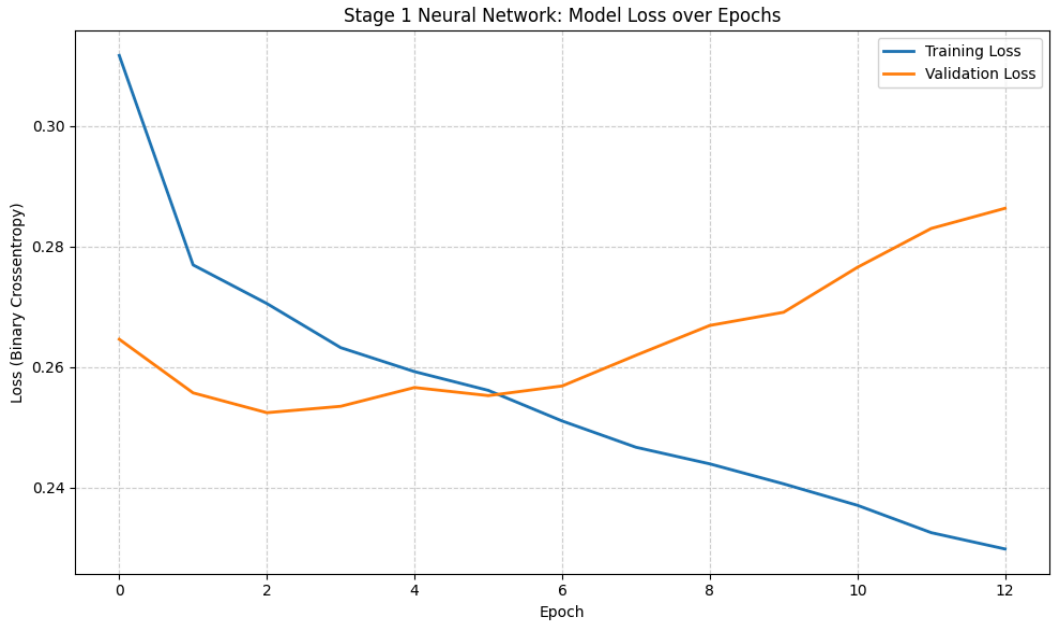


Confusion matrices are included to provide a detailed view of true positives, true negatives, false positives, and false negatives for each model and stage.

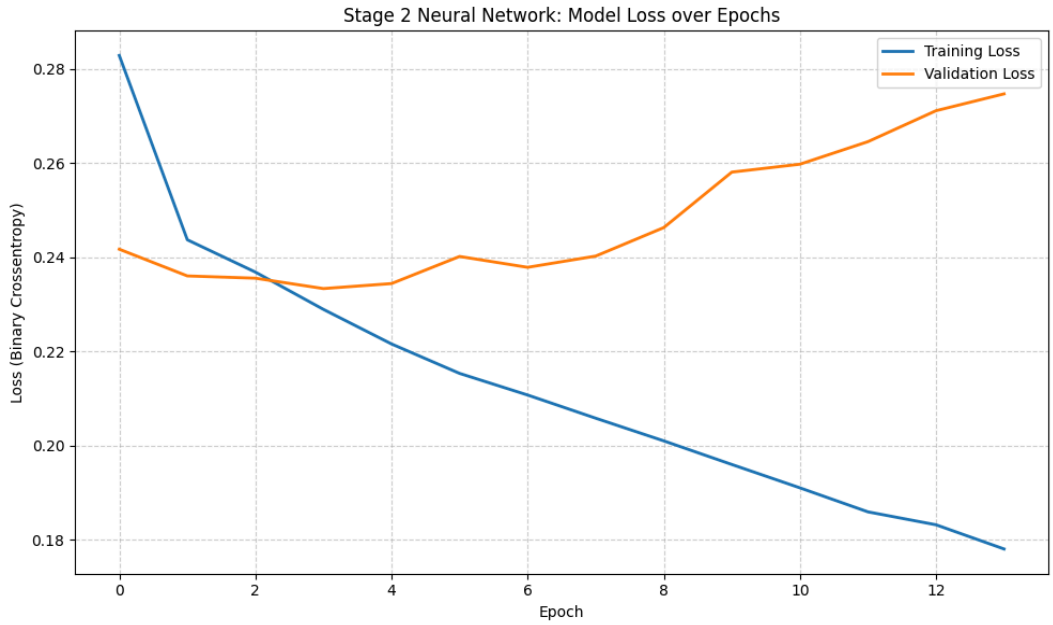
C3: Training history plots for the Neural Network

Loss curves for:

Stage 1 baseline



Stage 2 baseline



Neural Network loss curves are included to show model convergence behaviour and to identify potential overfitting.